Proyecto Final

Alejandro Coman, Alejandro Torres, David Sardiñas, Miguel Ubierna

1 - Contexto del proyecto

Idealista S.L. actúa como intermediario entre los ofertantes (particulares y agencias) y los demandantes. Entre sus principales objetivos se encuentran no solo el aumento del número de publicaciones, sino también la mejora de su calidad, con el fin de atraer a un mayor número de usuarios a la plataforma.

No obstante, en los últimos años, el volumen de anuncios ha crecido de forma considerable. Esto ha traído consigo un incremento de publicaciones con imágenes sin etiquetar o mal organizadas, lo que dificulta tareas como las siguientes:

- Mostrar en primer lugar las fotografías más relevantes,
- Ordenar automáticamente la galería de imágenes,
- Mejorar los filtros de búsqueda,
- Detectar anuncios de baja calidad.

Ante esta situación, se propone la implementación de un clasificador de imágenes que permita dar respuesta a los problemas anteriormente descritos. Esta solución contribuiría a mejorar la experiencia de los usuarios, ya que les permitiría localizar la información de interés de forma más rápida y eficiente. Además, también supondría una ayuda significativa para el equipo de operaciones, que podría trabajar de manera más eficiente al disponer de las fotografías previamente catalogadas.

En cuanto al valor esperado se esperan aportar los siguientes beneficios:

- Reducción de la necesidad de revisión y clasificación manual de imágenes por parte del personal, lo que permitiría agilizar los tiempos de publicación.
- Mejora de la experiencia de usuario mediante búsquedas avanzadas más eficientes y una mayor consistencia en el catálogo.
- Incremento del *Click-Through Rate* (CTR), ya que, si el sistema selecciona automáticamente la imagen más representativa del inmueble (por ejemplo, el salón) como foto de portada del anuncio, aumentará la probabilidad de que los usuarios hagan clic en él.

2 - Arquitectura del sistema

En este proyecto se ha adoptado una arquitectura modular de deep learning *end-to-end*, estructurada en tres capas principales. En primer lugar, se encuentra la capa de entrenamiento, en la que se construye, entrena y optimiza el modelo de clasificación de imágenes. A continuación, la capa de servicio, encargada de exponer el modelo a través de una API. Por último, la capa de presentación permite la interacción entre las capas anteriores y los usuarios finales.

2.1 - Capa de entrenamiento

La capa de entrenamiento es el componente en el que se define, entrena y optimiza el modelo. Este proceso se implementa en el script ***train.py***, donde se desarrolla prácticamente todo el flujo de entrenamiento a partir de las configuraciones establecidas en los archivos ***config.yaml*** y ***sweep.yaml.***

El procesamiento de los datos se realiza mediante el script ***dataset.py***, encargado de cargar las imágenes utilizando la clase ***ImageFolder*** de ***PyTorch***. Con el fin de mejorar la capacidad de generalización del modelo, se aplicaron distintas transformaciones al conjunto de entrenamiento mediante técnicas de data augmentation, como rotaciones, variaciones de color y borrado aleatorio.

Durante el entrenamiento, se monitorizaron las métricas obtenidas por los distintos modelos y se almacenó aquel que alcanzó la mayor precisión sobre el conjunto de validación en el archivo ***best_model.pth*.** Este modelo final es el que se utiliza posteriormente en la capa de servicio para alimentar la API y generar las predicciones.

2.2 - Capa de servicio

La capa de servicio del sistema se implementa mediante una *API REST* desarrollada con **FastAPI**, cuya función es exponer el modelo entrenado y permitir la realización de predicciones en tiempo real a partir de imágenes. Esta API actúa como intermediaria entre el modelo y el usuario final.

La API se estructura en varios endpoints principales:

- **GET /**: permite comprobar el estado del servicio y verificar que el modelo se ha cargado correctamente.
- **GET /classes**: devuelve la lista de clases que el modelo puede predecir. Esta información se obtiene del dataset utilizado durante el entrenamiento y facilita su uso en el frontend.
- **POST /predict**: es el endpoint principal, encargado de recibir una imagen y devolver la predicción del modelo. Internamente, la imagen se preprocesa utilizando las mismas transformaciones que en validación, se convierte en tensor y se introduce en el modelo en modo evaluación. A partir de la salida del modelo, se calculan probabilidades mediante softmax y se obtiene tanto la clase más probable como el top-5 de predicciones. La respuesta se devuelve en formato JSON, incluyendo la clase predicha, su probabilidad asociada y las cinco clases más probables.
- **POST /reload**: permite recargar el modelo desde disco sin necesidad de reiniciar el servidor. Esto resulta útil para actualizar el modelo tras generar un nuevo checkpoint durante el entrenamiento.

Este diseño basado en API permite una separación clara entre el modelo y la interfaz de usuario, facilitando su reutilización, mantenimiento y despliegue en distintos entornos.

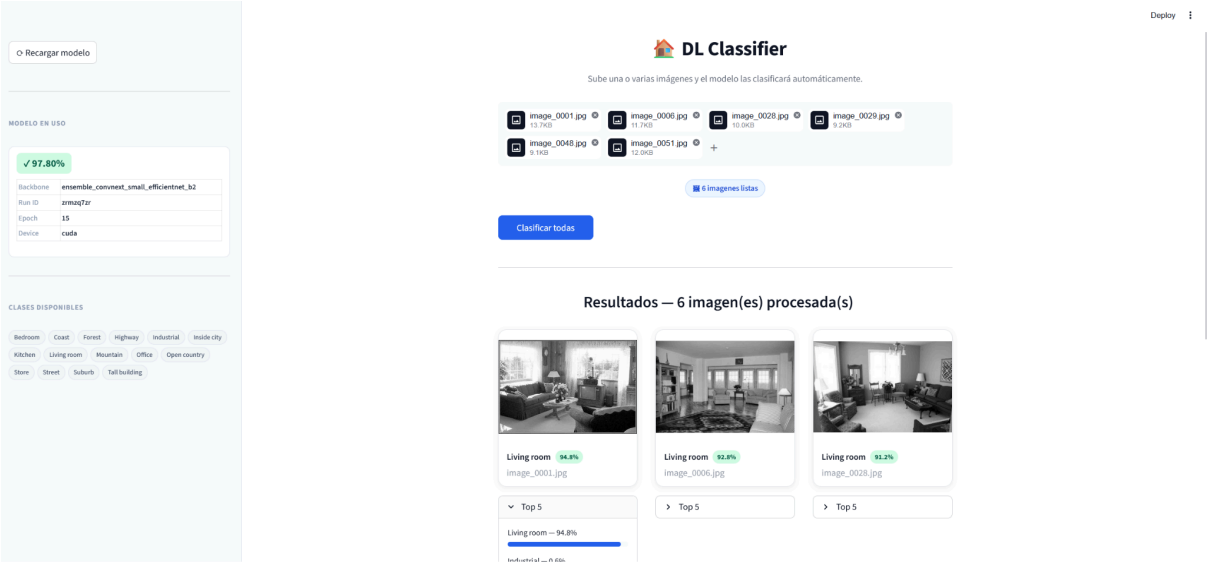
```
{
  "prediction": "Kitchen",
  "confidence": 0.9731,
  "top5": [
    { "class": "Kitchen",     "probability": 0.9731 },
    { "class": "Living room", "probability": 0.0142 },
    { "class": "Office",      "probability": 0.0087 },
    { "class": "Store",       "probability": 0.0031 },
    { "class": "Bedroom",     "probability": 0.0009 }
  ]
}
```


Ejemplo de respuesta del endpoint /predict

2.3 - Capa de presentación

Por último, la capa de presentación se implementa mediante una aplicación web desarrollada con Streamlit, que actúa como interfaz de usuario del sistema.

La interfaz permite al usuario cargar una o varias imágenes simultáneamente mediante un sistema de *file uploader*. Una vez seleccionadas, las imágenes se envían a la API de inferencia a través de peticiones HTTP, concretamente al endpoint **POST /predict**. Todo este proceso se realiza de forma transparente para el usuario, sin necesidad de interactuar directamente con la API.



Diseño final del frontend de la aplicación

Tras recibir la respuesta, la aplicación muestra los resultados de forma estructurada en una cuadrícula. Para cada imagen se visualiza:

- La imagen original
- La clase predicha por el modelo
- El nivel de confianza asociado a dicha predicción

- Un desglose de las cinco clases más probables (*top-5*), representadas mediante barras de progreso

Además, la aplicación incluye una barra lateral que proporciona información adicional sobre el sistema. En ella se muestra el estado de la API, así como detalles del modelo actualmente cargado, como el backbone utilizado, la precisión en validación o el identificador de la ejecución. También se incluye un listado de las clases disponibles, lo que facilita la interpretación de las predicciones.

3 - Modelling Approach

El enfoque de modelado adoptado se basa en el uso de técnicas de transfer learning aplicadas a modelos de Deep Learning pre entrenados, lo que permite obtener buenos resultados incluso con conjuntos de datos de tamaño limitado.
En concreto, se han utilizado arquitecturas modernas de clasificación de imágenes pre entrenadas en el dataset ImageNet, cómo ***EfficientNet*** y ***ConvNeXt***.

La estrategia de transfer learning empleada consiste en reutilizar el backbone de estos modelos (encargado de la extracción de características) y adaptar únicamente la parte final de la red al problema específico. Para ello, se reemplaza la capa de clasificación original por una nueva capa totalmente conectada cuyo número de salidas coincide con el número de clases del dataset (15 en nuestro caso). Además, se introduce una capa de dropout previa al clasificador con el objetivo de mejorar la regularización y reducir el sobreajuste.

El proceso de entrenamiento se realiza en dos fases diferenciadas:

- En la **primera fase**, se congelan los pesos del backbone pre entrenado y se entrena únicamente la capa de clasificación. Esta etapa permite adaptar rápidamente el modelo al nuevo problema sin alterar las representaciones generales aprendidas previamente.
- En la **segunda fase**, se descongela toda la red y se realiza un fine-tuning completo con una tasa de aprendizaje más baja. De este modo, el modelo puede ajustar progresivamente todas sus capas a las características específicas del dataset, mejorando su rendimiento final.

Para la optimización del modelo se utiliza el algoritmo ***AdamW***, combinado con un scheduler de tipo Cosine Annealing, lo que permite una reducción progresiva del learning rate durante el entrenamiento. Asimismo, se incorporan técnicas adicionales de regularización como el uso de weight decay y label smoothing, que contribuyen a mejorar la generalización del modelo.

Como mejora adicional, se ha implementado un enfoque de ensemble que combina las predicciones de dos modelos distintos: ***EfficientNet-B2*** y ***ConvNeXt-Small*** (ya que ambos modelos por separado eran los que mejor nos habían funcionado). Este ensemble se basa en un esquema de soft voting, en el que se calcula una media ponderada de las probabilidades predichas por cada modelo.Este enfoque permite reducir la varianza del sistema y aprovechar las fortalezas complementarias de ambas arquitecturas.

El mejor rendimiento en términos de métricas de validación se obtuvo mediante la estrategia de ensamble mencionada.

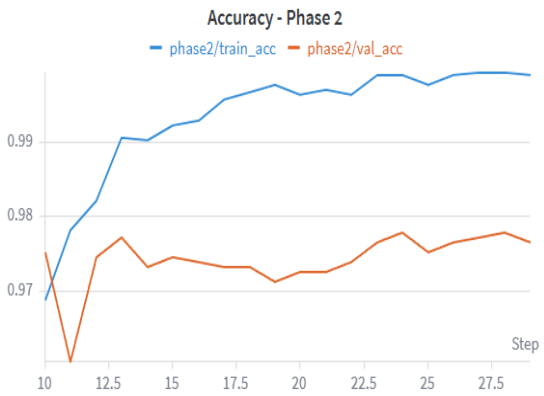
4 - Proceso de experimentación y métricas

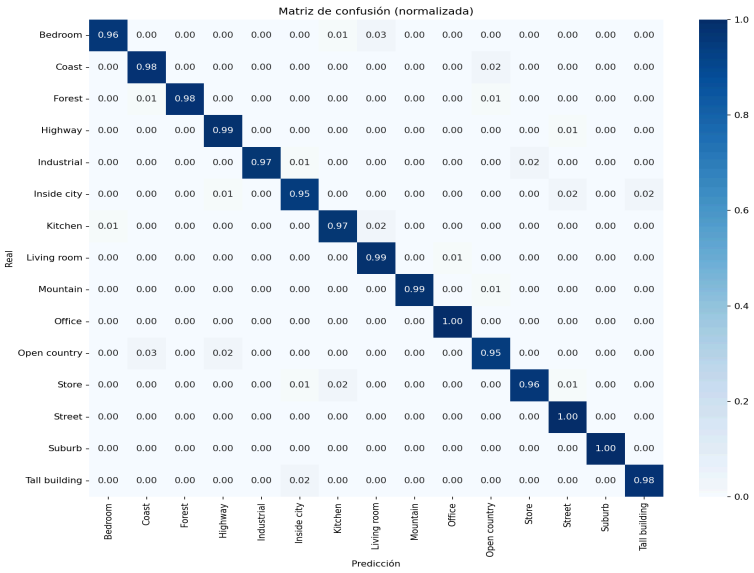
La herramienta Weights and Baises, nos ha permitido realizar un seguimiento sistemático de los diferentes entrenamientos y configuraciones probadas. Para cada una de las ejecuciones de cada uno de los miembros del grupo, se han ido registrando métricas como la pérdida y precisión en entrenamiento y validación así como otros hiperparámetros utilizados.

Gracias a esta plataforma hemos podido ver la evolución de nuestros modelos y trabajar paralelamente entre los miembros del equipo. A continuación se muestran las métricas obtenidas de los modelos que mejor nos han funcionado:

4.1 - Ensemble

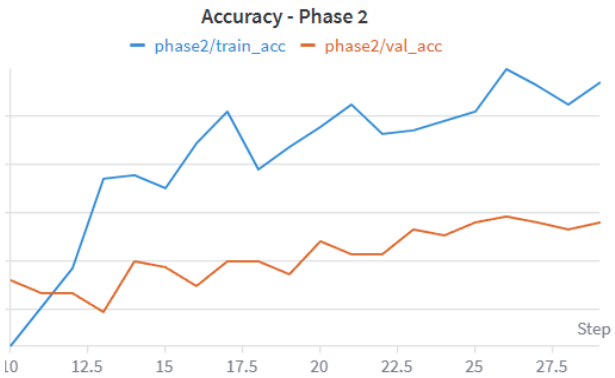
Es el modelo con el que hemos obtenido mejores resultados y con el que se hace la evaluación final en la aplicación.

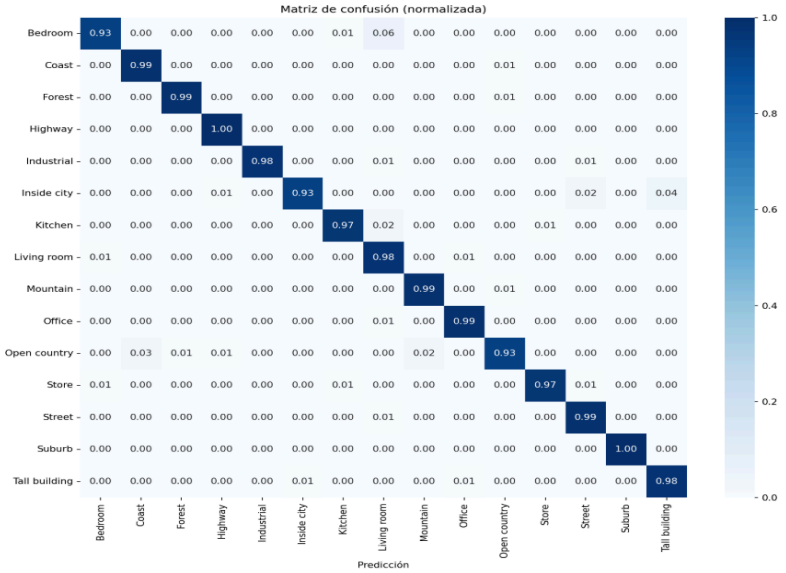




Izquierda: Accuracy en train y validación. Derecha: Matriz de confusión

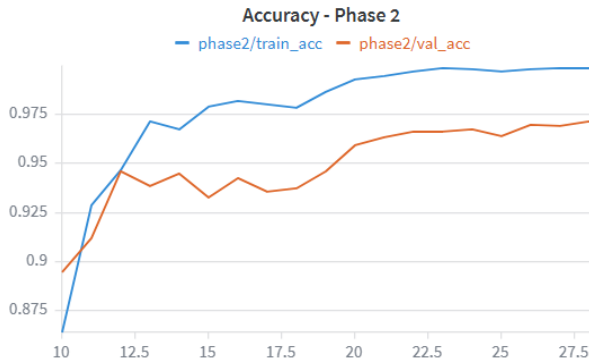
4.2 - EfficientNet-B2

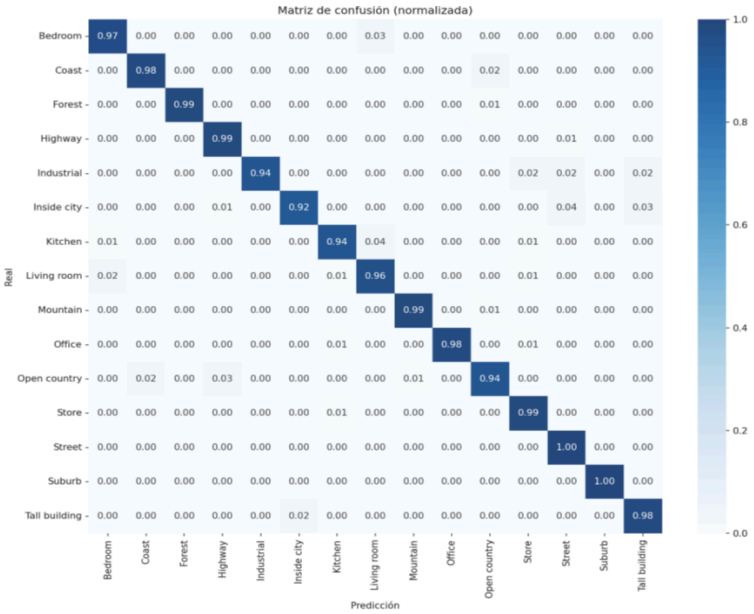




Izquierda: Accuracy en train y validación. Derecha: Matriz de confusión

4.3 - ConvNeXt-Small





Izquierda: Accuracy en train y validación. Derecha: Matriz de confusión

5 - Enlaces

- GitHub: https://github.com/AleComan/DL_Classifier
- W&B: https://wandb.ai/202513902-universidad-pontificia-comillas

Anotación: En el readme del repo de GitHub se detallan los pasos para desplegar las APIs (incluido el acceso a su documentación con swagger) y la aplicación StreamLit así como otros comandos e información importante.